

LifeSystem

32-Week Beginner-to-Production Major Project Roadmap

Project goal: Build an AI-powered personal guidance and progression system that helps people understand their current situation, choose the next useful action, improve across Physical, Educational, Mental Wellness, Emotional Wellbeing, and Career/Life Direction, and progress sustainably.

Core rule: build the decision engine first; use the LLM as one component, not the whole product. Wellness features are supportive and non-diagnostic.

How to use this roadmap

Each week has three jobs: **Learn** the minimum concept, **Build** one concrete feature, and **Prove** it with a visible deliverable. Spend roughly 60% of project time building, 25% learning, and 15% testing/documenting.

Suggested weekly rhythm: Monday learn + plan; Tuesday-Wednesday code; Thursday integrate; Friday test; Saturday document + Git commit/release note; Sunday review and fix the biggest issue.

Phase map

Phase	Weeks	Outcome
Phase 1 - Learn the foundations	Weeks 1-4	Python, Git, APIs, FastAPI, backend structure
Phase 2 - Build the data foundation	Weeks 5-8	SQL, PostgreSQL, authentication, user profiles
Phase 3 - Build the System core	Weeks 9-16	assessment, scoring, status, XP, quests, goals, evidence
Phase 4 - Make it intelligent	Weeks 17-20	adaptive difficulty, bottlenecks, next-best-action, explainability
Phase 5 - Make it supportive	Weeks 21-24	recovery mode, recommendations, safety, controlled AI
Phase 6 - Build the real UI	Weeks 25-28	React dashboard and complete core experience
Phase 7 - Validate and deploy	Weeks 29-31	ML experiment, testing, security, deployment
Phase 8 - Make it job-ready	Week 32	portfolio, report, presentation, interview preparation

32-week execution plan

Week 1: Project foundation

Learn: Learn Git basics, Python project structure, virtual environments. Create GitHub repo and folder structure.

Build / prove: Repo created; README; first commit; environment works.

Week 2: Python refresh

Learn: Revise functions, classes, lists/dicts, exceptions, modules, file I/O. Write 10 small exercises.

Build / prove: Python practice folder + notes.

Week 3: FastAPI basics

Learn: Learn routes, GET/POST, JSON, request/response, validation. Build health-check API.

Build / prove: Working FastAPI service + /docs.

Week 4: REST + backend structure

Learn: Learn routers, services, schemas, configuration, environment variables. Refactor backend.

Build / prove: Clean backend architecture.

Milestone gate: Do not move on until the listed deliverable works end-to-end and is committed to Git.

Week 5: SQL fundamentals

Learn: Learn tables, keys, CRUD, SELECT, WHERE, JOIN, GROUP BY. Practice daily.

Build / prove: SQL practice database.

Week 6: PostgreSQL + ORM

Learn: Install PostgreSQL; learn SQLAlchemy and migrations. Connect backend to database.

Build / prove: Users table + DB connection.

Week 7: Authentication

Learn: Learn password hashing, JWT/session concepts, protected routes. Implement registration/login.

Build / prove: Working auth flow.

Week 8: User profile

Learn: Create user profile and goal models. Store age range, goals, preferences, availability, domains.

Build / prove: Profile API + DB schema v1.

Milestone gate: Do not move on until the listed deliverable works end-to-end and is committed to Git.

Week 9: Assessment design

Learn: Design onboarding questions for physical, educational, mental-wellness, emotional, career/life direction. Define safe boundaries.

Build / prove: Assessment question bank + scoring spec.

Week 10: Assessment engine

Learn: Build assessment API, validation, scoring, and profile summary.

Build / prove: User can complete assessment and receive baseline scores.

Week 11: Status system

Learn: Create status model: level, XP, rank, domain scores, balance score, recent trends.

Build / prove: First System Status endpoint.

Week 12: XP + levels

Learn: Design transparent XP formulas, level thresholds, anti-cheat logic, daily limits.

Build / prove: Leveling engine + unit tests.

Milestone gate: Do not move on until the listed deliverable works end-to-end and is committed to Git.

Week 13: Quest model

Learn: Design quest schema: objective, difficulty, domain, duration, prerequisites, evidence, reward.

Build / prove: Quest database + CRUD APIs.

Week 14: Rule-based quest generator

Learn: Before AI, build deterministic quest generation from rules and user state.

Build / prove: Personalized quest generator v1.

Week 15: Goal roadmap engine

Learn: Convert long-term goals into milestones, skills and prerequisite chains.

Build / prove: Roadmap feature v1.

Week 16: Evidence engine

Learn: Track quizzes, coding tasks, study sessions, exercise logs, habit evidence and outcomes.

Build / prove: Evidence tables + scoring hooks.

Milestone gate: Do not move on until the listed deliverable works end-to-end and is committed to Git.

Week 17: Adaptive difficulty

Learn: Calculate success rate, consistency and overload signals. Increase/decrease quest difficulty.

Build / prove: Adaptive quest engine v1.

Week 18: Bottleneck detection

Learn: Find high-impact blockers, not just lowest scores. Create explainable priority logic.

Build / prove: Bottleneck + priority engine.

Week 19: Next Best Action

Learn: Combine goal, bottleneck, current state, time and recent outcomes to select one primary action.

Build / prove: NBAction engine v1.

Week 20: Explainability

Learn: Add "Why this quest?" with traceable factors and confidence/rationale.

Build / prove: Explanation endpoint + UI mock.

Milestone gate: Do not move on until the listed deliverable works end-to-end and is committed to Git.

Week 21: Recovery mode

Learn: Build state check-ins for tired/stressed/overwhelmed/low-energy. Switch from growth mode to recovery/support mode.

Build / prove: Recovery Mode logic + safeguards.

Week 22: Content recommendation

Learn: Design recommendation catalog for walks, breathing, music, videos, light entertainment, journaling and social connection.

Build / prove: Recommendation rules + content metadata.

Week 23: Safety layer

Learn: Add boundaries: no diagnosis, no treatment claims, crisis/escalation language, refusal/safe redirect patterns, content filters.

Build / prove: Safety policy module + test cases.

Week 24: LLM integration

Learn: Use LLM for wording, summarization and option generation under strict schemas; backend remains the authority.

Build / prove: Structured AI service with fallbacks.

Milestone gate: Do not move on until the listed deliverable works end-to-end and is committed to Git.

Week 25: Frontend foundation

Learn: Learn React basics; routing, state, forms, API calls. Build login and onboarding screens.

Build / prove: React app connected to backend.

Week 26: System dashboard

Learn: Build status screen, domain cards, XP/level, today's quest, progress charts.

Build / prove: Usable dashboard v1.

Week 27: Guidance experience

Learn: Build daily guidance, quest detail, completion/evidence submission, "why this" screen.

Build / prove: Complete core user journey.

Week 28: Recovery + personalization UI

Learn: Build check-in, recovery mode, recommendations, preferences and content feedback.

Build / prove: Recovery flow + feedback.

Milestone gate: Do not move on until the listed deliverable works end-to-end and is committed to Git.

Week 29: Analytics + ML experiment

Learn: Collect synthetic/anonymized development data. Explore prediction of quest success/overload. Train a simple baseline model only if justified.

Build / prove: ML experiment notebook + evaluation.

Week 30: Testing + security

Learn: Unit, integration and API tests; input validation; auth tests; rate limits; secret handling; privacy review.

Build / prove: Test suite + security checklist.

Week 31: Deployment

Learn: Deploy backend/database/frontend. Set environment variables, logging, health checks, backups.

Build / prove: Public demo deployment.

Week 32: Portfolio + major project

Learn: Polish UI, documentation, architecture diagram, demo video, report, presentation, resume bullets and interview questions.

Build / prove: Final release v1.0 + report + demo.

Milestone gate: Do not move on until the listed deliverable works end-to-end and is committed to Git.

Detailed “how to build” sequence

A. Start small

Create the backend first. Make one endpoint work, then add one model, then one feature. Avoid building the UI, AI and database simultaneously.

B. Design before coding

For every feature write: user problem, input, output, business rules, database fields, API endpoint, success criteria, edge cases.

C. Keep AI bounded

The backend decides allowed actions and safety rules. The LLM can draft wording, summarize evidence, and propose alternatives inside a schema. Validate all AI output before storing or showing it.

D. Make progress measurable

Every major domain needs observable evidence: quiz results, problem completion, routine consistency, self-check-ins, goal milestones or user feedback.

E. Build explainability

Store the factors behind each priority decision. This makes “Why this quest?” possible and gives you a strong interview story.

F. Test with synthetic users first

Create 20-50 fictional profiles representing different situations. Verify that the System does not recommend identical plans to everyone.

G. Deploy early

Deploy a basic version before the final month. Fixing production problems is part of the project, not an afterthought.

Recommended technical stack

Layer	Recommended	Why
Frontend	React + JavaScript/TypeScript	Professional web UI and component model
Backend	Python + FastAPI	Matches your Python background; fast API development
Database	PostgreSQL	Strong SQL skills and production-ready relational storage
ORM/Migrations	SQLAlchemy + Alembic	Maintainable data models and schema changes
AI	LLM API with structured JSON output	Language/recommendation support without giving away business logic
ML	scikit-learn	Simple interpretable baseline models later
Charts	Recharts or Chart.js	Progress and trend visualization
Testing	Pytest + API tests	Reliable backend and decision logic
Deployment	Docker + a cloud platform	Repeatable deployment and portfolio demonstration

What the final system should contain

1. Onboarding + assessment

Collect minimum necessary profile information, goals, preferences, availability and domain self-assessments.

2. Status screen

Level, XP, rank, domain scores, trends, balance and current system priority.

3. Goal roadmap

Long-term goal -> milestones -> skills -> evidence -> quests.

4. Next Best Action Engine

Select the most useful next action from goals, bottlenecks, current state, available time and recent performance.

5. Adaptive quest engine

Adjust difficulty using success rate, consistency, workload and feedback.

6. Recovery & Support Mode

When the user reports tiredness, stress, anxiety or low energy, shift toward low-risk recovery/support actions rather than punishment.

7. Content recommendations

Offer suitable videos, music, short activities, journaling or entertainment using preference + context filters.

8. Evidence-based progress

Use actual learning/performance evidence where possible instead of checkbox-only XP.

9. Explainable recommendations

Show why a quest or priority was selected.

10. Safety/privacy

No diagnosis or treatment claims; clear escalation for serious/urgent distress; data minimization, authentication and secure storage.

What to postpone until the core works

Do **not** start with mobile apps, wearables, complex deep learning, social networks, live coaching, elaborate anime effects, or dozens of recommendation APIs. Those are expansion features. The major-project score should come from a solid working system first.

Job-focused outcome by the end

You should be able to show a live demo, GitHub repository with clean commits, architecture diagram, API docs, database schema, tests, deployment, a short demo video, a final report, and 4-6 strong resume bullets. In interviews, be ready to explain one real technical decision from every layer: database design, API design, scoring logic, adaptive algorithm, AI guardrails, testing, and deployment.

Weekly checklist template

Day	Action
Monday	Understand the week's concept; write a one-page note; define the feature and acceptance criteria.
Tuesday	Implement the smallest working version.
Wednesday	Connect the feature to the database/API and handle invalid inputs.
Thursday	Integrate with the previous module; test at least 3 normal and 2 edge cases.
Friday	Fix bugs; add unit/API tests; clean code.
Saturday	Update README/docs, capture screenshots, commit with a meaningful message.
Sunday	Review: What works? What failed? What did I learn? What is the single biggest task next week?

Golden rule: every week must end with a working artifact. Do not measure progress by how many tutorials you watched; measure it by what the system can now do.

Change management

This roadmap is Version 1.0. New ideas can be added later through versioned changes: record the user problem, benefit, complexity, safety/privacy impact, dependencies and the week where it belongs. Keep the core architecture stable unless the new requirement truly justifies a redesign.